

FLOODTAIL — System Architecture & Problem Alignment

Authority: [Team_A_Nairobi_Problem_Statement.docx](#) (Nairobi Urban Flood Challenge · Team A)

Product: Reinsurance-oriented Nairobi County **pluvial** (surface-water) flood catastrophe prototype

Audience: Judges, underwriters, portfolio managers, and technical reviewers

This document describes the problem FLOODTAIL solves, the system architecture, how a run executes, the technologies used, and how each hackathon objective is met.

1. The problem we are solving

1.1 Context (from the problem statement)

Insurance protects policyholders against covered losses. **Reinsurance** protects insurers (cedants) when one event can produce many claims at once — **accumulation**. Nairobi's drainage has not kept pace with urban growth; county mapping has identified dozens of flood-prone neighbourhoods. Flooding is Kenya's most frequent damaging natural peril, yet local flood risk is still largely managed with underwriter judgement and broad global hazard layers rather than a systematic local model.

Global CAT vendors build full pipelines, but their models rely on proprietary claims data that is unavailable here. Team A's brief is therefore:

Build a working AI-powered flood model for Nairobi — from identifying flood risk to estimating losses for different flood events — using **open / synthetic data**, covering the classic CAT chain, and using AI in a way that **materially changes** the result.

1.2 What a CAT model must answer

Stage	Question
Hazard	Where does flooding happen, and how severe is it?
Vulnerability	How much damage does a property suffer at a given severity?
Exposure	What assets are at risk, and what are they worth?
Financial engine	What insured losses result, and how do they vary by rarity?

Underwriters need more than a single “expected loss”: they need an **exceedance probability (EP) / return-period curve** — e.g. what loss is associated with a roughly 1-in-100-year severity — so they can budget, set capital, and judge concentration.

1.3 Current vs desired state

Current state (problem statement)	Desired state (FLOODTAIL)
No locally calibrated Kenyan flood CAT model in-house	Working end-to-end Nairobi pluvial prototype
Judgement + global hazard references	Documented proxy hazard + trained ML scores
No systematic losses-by-return-period tool	Deterministic EP / RP curve + discrete AAL
No public measured pluvial depths for Nairobi	Honest proxy rasters + synthetic labels, clearly labelled
AI often bolted on as commentary	Predictive ML + agentic AI that change scores / exposure / briefing

1.4 Constraints we honour

- **Synthetic exposure only** — no real client portfolios
- **Proxy hazard** — not gauge-validated hydrology; starter proxy aligns with ~12/24 named hotspots
- **Vulnerability** — JRC/Huizinga-adapted curves (no Kenya claims calibration)
- **Discrete scenarios / RP map** — not a full stochastic catalogue
- Out of scope: treaty layers, multi-peril, full rainfall–runoff hydrology

2. How FLOODTAIL solves it

FLOODTAIL realises the required chain:

Hazard → Vulnerability → Exposure → Financial engine → EP / return-period curve

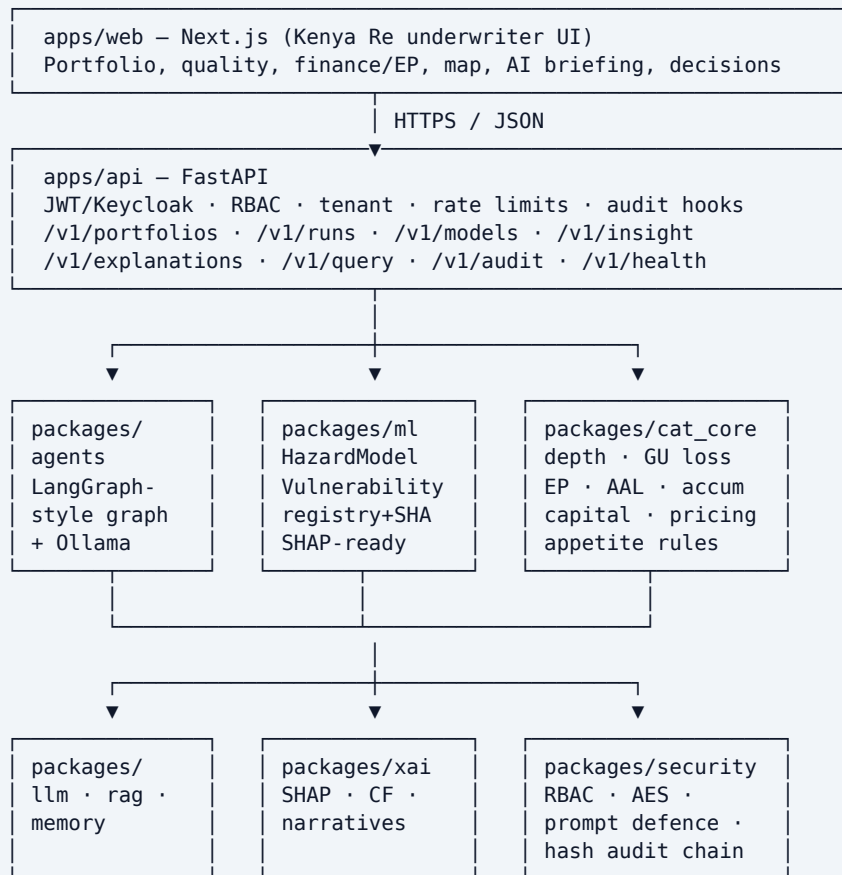
with a locked **AI posture (option 3)**:

1. **Predictive ML** — trained, versioned HazardModel and VulnerabilityModel change numeric scores and damage ratios.
2. **Agentic AI (Ollama)** — ingest, enrichment messaging, free-text exposure, briefings, Q&A, human-gate summaries.
3. **Deterministic core (packages/cat_core)** — depth mapping, ground-up loss, EP, AAL, accumulation, pricing, appetite rules. **LLMs never invent money numbers.**

Honest labelling travels with every payload (synthetic=True , data_labels , aal_caveat , assumptions version). Humans remain in the loop for material decisions; AI recommends, humans approve.

3. System architecture

3.1 Layered view



Data: Nairobi_Data/ · models/{hazard,vulnerability}/ · Postgres (pgvector)

Infra: Docker Compose – api · postgres · keycloak · ollama

Deploy: Vercel (web) · Contabo VPS (API + Ollama + Postgres) – see DEPLOYMENT.md

3.2 Package responsibilities

Package	Role	LLM?
apps/web	Results interface: EP curve, exposure, class breakdown, AI narrative, decisions	No (client)
apps/api	HTTP contract, auth, orchestration endpoints, run/portfolio store	No
packages/agents	Stage graph: schema/PII/enrich, free-text exposure, gates, briefing, query	Yes (Ollama)
packages/ml	Train + infer hazard & vulnerability; feature build; model registry	No
packages/cat_core	Depth, loss, EP, AAL, accumulation, capital band, appetite	No
packages/xai	SHAP, counterfactuals, validated narrative helpers	Partial
packages/security	RBAC, encryption, prompt defence, output number allowlist, audit chain	No
packages/llm	Ollama client	Yes
packages/rag / memory	Knowledge ingest/retrieve; long-term memory (optional)	Assist

3.3 Hard authority boundary

May change	Owner
Exposure rows, enrichment notes, briefings, Q&A prose	Agentic AI
Per-tier hazard scores, damage ratios	Predictive ML
Depth (m), GU loss, EP points, AAL, premiums, accumulation, appetite codes	cat_core only

If an LLM invents a number not in the frozen allowlist, output validation rejects it and the system falls back to a template + audit event. Critical stage failures halt the run; non-critical failures (e.g. briefing) warn and continue.

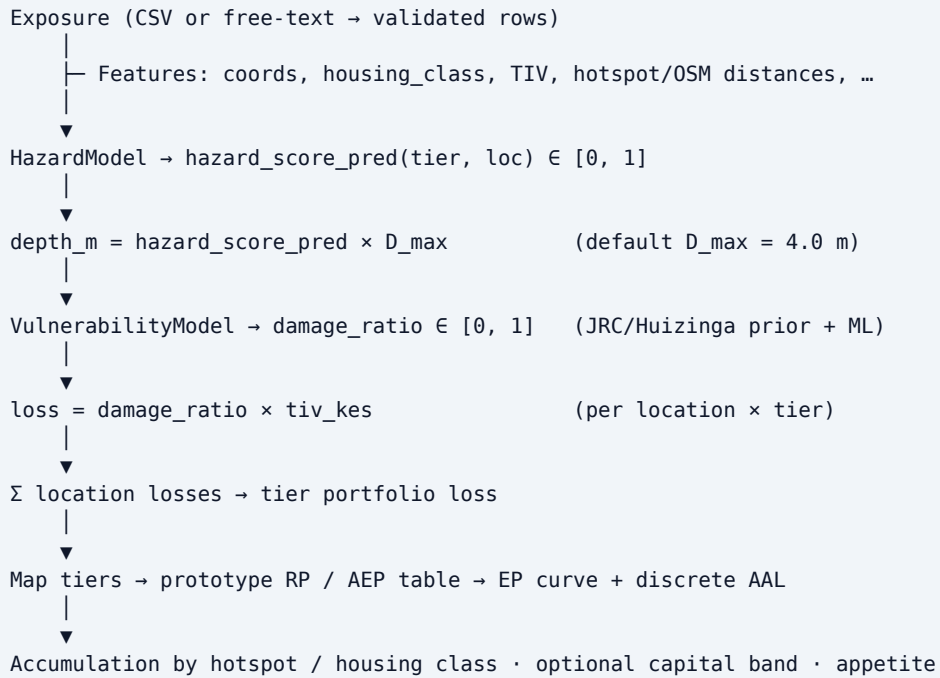
4. Logic flow (end-to-end run)

4.1 Pipeline stages

Aligned with [AGENTS.md](#) and the problem statement's Steps 1–6:

1. SchemaValidate / PII / Geocode / Enrich	[agentic + rules]	CRITICAL
2. FreeTextExposure (optional)	[agentic LLM]	non-critical
3. HumanGate_1 (if enabled)	[human + messaging]	configurable
4. PredictHazard	[predictive ML]	CRITICAL
5. PredictVulnerability	[predictive ML]	CRITICAL
6. Financial + EP + Accumulation	[deterministic]	CRITICAL
7. XAI + Briefing	[ML explain + LLM]	non-critical
8. Governance / Audit / HumanGate_2	[rules + audit]	decision gate

4.2 Numeric path (problem-statement Steps 1–4)



Prototype return-period map (documented assumption, not a stochastic catalogue):

Tier	Assumed RP (years)	Approx. AEP
common	5	0.20
occasional	20	0.05
moderate	50	0.02
severe	100	0.01
extreme	250	0.004

AAL honesty: metrics carry `aal_method=discrete_ep_sum` and an `aal_caveat` — discrete $\Sigma(\text{AEP} \times \text{tier_loss})$, not a vendor stochastic mean.

4.3 Typical API / operator flow

```
# 1. Bring stack up
docker compose up --build
# or: uvicorn apps.api.main:app --reload --port 8000
#     cd apps/web && npm run dev

# 2. Ingest portfolio (built-in Nairobi or CSV upload)
POST /v1/portfolios

# 3. Train & pin ML (materially changes scores)
POST /v1/models/hazard/train
POST /v1/models/vulnerability/train

# 4. Run pipeline
POST /v1/runs

# 5. Underwriter-facing outputs
GET /v1/runs/{id}/insight      # houses, set-aside band, recommendation
GET /v1/runs/{id}/metrics     # EP, AAL, data_labels, aal_caveat
GET /v1/runs/{id}/narrative   # validated Ollama briefing
GET /v1/audit                 # hash chain integrity
```

Health: GET /v1/health · OpenAPI: <http://localhost:8000/docs> .

5. Technologies used

Layer	Technology
Frontend	Next.js, React, TypeScript, Tailwind, MapLibre (flood map)
API	FastAPI (Python), Pydantic schemas, Uvicorn
Agentic orchestration	LangGraph-style state machine (packages/agents), LangChain tools
LLM runtime	Ollama (default instruct model e.g. qwen2.5:3b-instruct); template degrade if down
Predictive ML	XGBoost / LightGBM (gradient-boosted trees), joblib artifacts, SHA-256 registry
Geospatial	GeoTIFF proxy rasters, optional rasterio sampling, OSM distance features
CAT math	Pure Python deterministic modules in packages/cat_core
Explainability	SHAP (TreeExplainer), counterfactuals, number-allowlist narrative validation
Auth / tenancy	Keycloak (OIDC JWT), RBAC roles, tenant scoping
Data stores	Postgres + pgvector (RAG); file/memory run store; model artifact dirs
Security	Prompt injection defence, AES-GCM for PII, irreversible ID hashing, audit hash chain
Infra / deploy	Docker Compose; Vercel (web); Contabo VPS + Nginx (API)

6. Data assets (starter kit)

Asset	Role	Honest label
exposure_nairobi_with_hazard.csv	600 synthetic buildings + 5-tier scores	Synthetic
exposure_nairobi_synthetic.csv	Same locations without scores (raster / enrich path)	Synthetic
nairobi_pluvial_proxy_{common... extreme}.tif	Susceptibility rasters 0–1	Proxy (not measured depth)
nairobi_hotspots_geocoded.csv	24 government-named hotspots	QA ≈ 12/24 elevated

All exposure rows are stamped `synthetic=True`. Hazard ML labels = proxy + hotspot uplift + optional OSM drainage uplift — **not** flood gauges. Vulnerability labels derive from JRC/Huizinga-adapted priors — **not** Kenya claims.

7. How the system meets the project objectives

Mapped directly to **§2 Objectives**, **§5 Desired State**, **§7 Scope**, and **Steps 1–6** of the Team A problem statement.

Objective A — End-to-end flood loss pipeline

Problem-statement step	FLOODTAIL implementation
Ingest hazard data	Built-in CSV scores and/or GeoTIFF sampling; enrichment features
Apply vulnerability functions	JRC/Huizinga-adapted priors + VulnerabilityModel inference
Assess losses per property	<code>loss = damage_ratio × tiv_kes</code> per location × tier
Aggregate by return period	Tier losses mapped to prototype RP / AEP table
Financial engine → EP curve	<code>packages/cat_core</code> EP + discrete AAL + accumulation

Met: Full Hazard → Vulnerability → Exposure → Financial → EP path in one run (`POST /v1/runs`).

Objective B — AI that materially enhances the result

The brief requires AI that **changes** the model, not only describes it. FLOODTAIL does this in multiple places:

AI use (problem statement examples)	How FLOODTAIL delivers material change
Improve hazard / vulnerability	Trained HazardModel & VulnerabilityModel change predicted scores and damage ratios → change GU loss and EP
Free-text → structured exposure	FreeTextExposure agent (Ollama) emits schema-validated rows stamped synthetic → portfolio composition changes
Risk briefing from model output	Briefing / narrative over frozen allowlisted metrics (prose only; numbers from <code>cat_core</code>)
Hazard layer signals	Enrichment (hotspots, OSM, agent notes) feeds features into ML

Evidence of material change: pin a new model version → baseline-vs-predicted deltas stored in run lineage; free-text ingest alters location count / TIV before the financial engine.

Objective C — Return-period / EP curve for non-modellers

- Metrics API and Next.js finance views show loss at key RPs and an EP chart.
- Insight endpoint summarises insured houses, set-aside band (floor / central / ceiling), and `ACCEPT` / `REVIEW` / `ESCALATE`.
- Plain-language narrative available after validation.

Met: EP curve is a first-class underwriter output, not an engineer-only artifact.

Objective D — Clearly state assumptions and synthetic / proxy use

- `ASSUMPTIONS.md` register with `SUPPLIED` / `VERIFIED` / `BENCHMARK` / `PROTOTYPE` taxonomy
- Every metrics payload: `assumptions_version`, `data_labels`, `aal_caveat`
- UI banners / prototype badges for synthetic exposure and proxy hazard
- Model card documents training-label honesty

Met: Placeholder or proxy numbers are not presented as real observations.

Objective E — Results interface for stakeholders

Problem statement stakeholders → FLOODTAIL surfaces:

Stakeholder	What they get
Underwriters & risk analysts	Insight, EP table/curve, narrative, approve with reason
Portfolio / exposure managers	Accumulation by hotspot and housing class; map views
Hackathon judges	Working demo + <code>ASSUMPTIONS</code> / <code>METHODOLOGY</code> / <code>MODEL_CARD</code>
County / disaster bodies	Hotspot-aware views (prototype; not operational authority)
Cedants & brokers	Faster, consistent RP loss views (prototype quote support)

Minimum UI checklist from Step 6: total exposure · key RP losses · EP curve · housing-class breakdown
· AI feature output — covered in `apps/web` + API.

Desired-state checklist (§5)

Desired capability	Status in FLOODTAIL
Ingests a hazard dataset	Yes — CSV / GeoTIFF / built-in Nairobi
Documented, sourced vulnerability function	Yes — JRC/Huizinga-adapted + <code>MODEL_CARD</code> / <code>ASSUMPTIONS</code>
Synthetic portfolio → loss + RP curve	Yes — 600-building starter + EP
AI that actually changes output	Yes — ML predictions and/or free-text exposure
Interface distinguishes real vs assumed	Yes — <code>data_labels</code> , caveats, banners

In-scope / out-of-scope discipline (§7)

In scope	Out of scope (honoured)
Proxy hazard ingestion	Treaty structuring / layers
Documented depth–damage	Real policy / claims integration
Synthetic exposure	Multi-peril aggregation
Loss engine + EP curve	Full physical pluvial hydrology
Material AI stage	—
Non-modeller results UI	—

8. Governance, honesty, and failure modes

- **Human-in-the-loop:** optional gate before ML/financial; mandatory justification on final approve.
- **RBAC:** underwriter vs data_scientist vs auditor roles (e.g. underwriters cannot train models).
- **Audit:** SHA-256 hash chain over upload, train, run, narrative, decide; `chain_valid` exposed.
- **Degradation:** Ollama down → templates; ML + `cat_core` still complete. Model hash mismatch → predict stages **FAILED**.
- **Prototype boundary:** not production authority to bind real portfolios; not gauge-validated hydrology.

9. Related documents

Doc	Purpose
Team_A_Nairobi_Problem_Statement.docx	Hackathon problem authority
AGENTS.md	Stage list and halt rules
METHODOLOGY.md	CAT + ML + agent methodology
ASSUMPTIONS.md	Assumptions register
MODEL_CARD.md	Model suite card
EXPLAINABILITY.md	SHAP, CF, narrative validation
RISK_GOVERNANCE.md	Gates, RBAC, audit
DEPLOYMENT.md	How to run in production
DEMO_SCRIPT.md	Judge walkthrough

10. One-sentence summary

FLOODTAIL turns the Team A brief — *build an honest Nairobi pluvial CAT pipeline with AI that changes the answer* — into a runnable system: synthetic exposure and proxy hazard in, trained ML and agents in the middle, deterministic loss and EP out, with a Kenya Re–facing UI and audit trail so underwriters and judges can see both the numbers and their limits.

